

Implantación de un Entorno Kubernetes Local con Aplicación Web Escalable y Monitoreo en Tiempo Real

Memoria Técnica del Proyecto Intermodular

Ciclo Formativo	2.º ASIR – Administración de Sistemas Informáticos en Red
Módulos	SRI (Servicios de Red e Internet), SAD (Seguridad y Alta Disponibilidad), ASO (Administración de Sistemas Operativos)
Curso	2025–2026
Tutora	Juan Ignacio Benítez
Equipo	Adrián Boza Suárez · Santiago Pérez Cano · Sergio López Pérez
Repositorio	https://github.com/adrianboza2/proyecto-intermodular-asir

Índice

Memoria Técnica del Proyecto Intermodular.....	1
1. Resumen Ejecutivo.....	3
2. Objetivos.....	4
3. Stack Tecnológico	5
4. Arquitectura del Sistema	6
5. Fases de Implementación	8
5.1. Fase 1: Infraestructura y Aplicación Web.....	8
5.2. Fase 2: Monitorización	11
1. Auto-descubrimiento de pods (.....	11
5.3. Fase 3: Seguridad de Red	13
6. Resultados y Verificación	14
6.1. Despliegue de la aplicación web.....	14
6.2. Acceso desde el navegador.....	14
6.3. Prometheus targets UP	14
6.4. Grafana con datasource conectado.....	14
6.5. Dashboard de Grafana con métricas en tiempo real.....	15
6.6. NetworkPolicies aplicadas.....	16
6.7. Test de conectividad	17
6.8. Prueba de alta disponibilidad	18
7. Problemas Encontrados y Soluciones	19
8. Relación con los Módulos de ASIR	20
Servicios de Red e Internet (SRI).....	20
Seguridad y Alta Disponibilidad (SAD).....	20
Administración de Sistemas Operativos (ASO)	20
9. Conclusiones	21
10. Mejoras Futuras	22
11. Anexo: Evidencias	23
12. Anexo: Glosario de Términos	24

1. Resumen Ejecutivo

Este proyecto consiste en la implantación de un clúster Kubernetes local mediante **Minikube** sobre una máquina virtual **Ubuntu Server 24.04 LTS** ejecutada en **VirtualBox**. El sistema despliega una aplicación web **Nginx** con alta disponibilidad (2 réplicas), un stack completo de monitorización con **Prometheus y Grafana**, y políticas de seguridad de red que siguen el modelo **zero-trust**.

Todo el proyecto se gestiona mediante **infraestructura como código (IaC)**: los manifiestos YAML están versionados en Git, lo que garantiza reproducibilidad, auditoría y reversibilidad. El proyecto está diseñado para ejecutarse en un portátil con recursos limitados (2 GB RAM, 2 CPUs para Minikube), simulando a pequeña escala el flujo de trabajo de un equipo DevOps en una empresa tecnológica.

Aportaciones Originales

- **Integración multidisciplinar:** El proyecto conecta conocimientos de los tres módulos de 2.º ASIR (SRI, SAD, ASO) en un caso práctico unificado.
- **Modelo zero-trust educativo:** Implementación completa de 5 NetworkPolicies que aíslan el tráfico entre servicios, documentando cada regla y su justificación.
- **Auto-descubrimiento de métricas:** Prometheus configurado con `kubernetes_sd_configs` para detectar automáticamente pods con anotaciones de scraping, sin configuración manual de targets.
- **Reproducibilidad total:** Cualquier alumno de ASIR puede clonar el repositorio y tener el mismo entorno funcionando con 4 comandos.

2. Objetivos

Objetivo General

Implementar un entorno Kubernetes funcional que despliegue una aplicación web con monitorización y seguridad, utilizando exclusivamente herramientas gratuitas y ejecutable en un ordenador personal.

Objetivos Específicos

ID	Objetivo	Responsable	Módulo asociado
OE1	Configurar un clúster Kubernetes con Minikube sobre Ubuntu Server en VirtualBox	Adrián	ASO
OE2	Desplegar una aplicación web Nginx con alta disponibilidad (2 réplicas) y auto-recuperación mediante health checks	Adrián	SRI
OE3	Implementar un sistema de monitorización con Prometheus y Grafana	Santiago	ASO
OE4	Aplicar políticas de seguridad de red con modelo zero-trust mediante NetworkPolicies	Sergio	SAD
OE5	Verificar el correcto funcionamiento mediante pruebas de conectividad, alta disponibilidad y monitorización	Todo el equipo	SRI, SAD, ASO
OE6	Documentar todo el proceso para su reproducción y evaluación	Sergio	—

3. Stack Tecnológico

Componente	Tecnología	Versión	Propósito
Hipervisor	VirtualBox	7.x	Virtualización de la VM
SO Invitado	Ubuntu Server	24.04 LTS	Sistema base para el clúster
Contenedores	Docker Engine	29.5.1	Runtime de contenedores para Minikube
Orquestador	Minikube	v1.38.1	Entorno Kubernetes de un solo nodo
API de Kubernetes	kubectl	v1.35.1	CLI para gestionar el clúster
App Web	Nginx	Latest	Servidor web de ejemplo
Métricas	Prometheus	Latest	Recopilación y almacenamiento de métricas
Dashboards	Grafana	10.4.3	Visualización de métricas con dashboards
Control de versiones	Git + GitHub	—	Versionado de manifiestos y documentación

Justificación de la elección tecnológica

- **Minikube frente a kind/k3s:** Minikube es el estándar en el ámbito educativo de ASIR por su facilidad de uso, documentación extensa y soporte multiplataforma. Es el que se ha utilizado durante el curso, lo que garantiza continuidad pedagógica.
- **Nginx frente a otras alternativas:** Es el servidor web más utilizado a nivel mundial (~30 % de los sitios web). Ligero, probado y con imagen oficial en Docker Hub. Su página de bienvenida por defecto sirve como indicador visual inmediato de que el despliegue funciona.
- **Prometheus + Grafana:** Es el stack de monitorización de facto en el ecosistema cloud-native. Prometheus está incubado por la CNCF (Cloud Native Computing Foundation) y Grafana es el estándar para dashboards. Ambos tienen imágenes oficiales ligeras y documentación extensa.
- **VirtualBox:** Gratuito, multiplataforma y el hipervisor utilizado en el aula de ASIR. Su modo NAT con port forwarding permite acceder a los servicios del clúster desde el host sin configuración de red compleja.

4. Arquitectura del Sistema

```
Windows 10/11 (Host)
├─
├─ VirtualBox (NAT + Port Forwarding)
│   └─ Ubuntu Server 24.04 LTS
│       │   Recursos: 4 GB RAM, 2 vCPU, 25 GB disco
│       │   Usuario: sergio / adrian
│       │   └─ Docker Engine (runtime de contenedores)
│           │   └─ Minikube (Kubernetes 1 nodo)
│               │   Recursos asignados: 2 GB RAM, 2 CPUs
│               │   └─ Namespace: default
│                   │   └─ [Deployment] nginx-web (2 réplicas)
│                       │   └─ Pod: nginx-web-xxx (app: nginx-web)
│                           │   └─ Pod: nginx-web-yyy (app: nginx-web)
│                               │   └─ LivenessProbe: HTTP GET / :80 (cada 10s)
│                                   └─ ReadinessProbe: HTTP GET / :80 (cada 5s)
│                   └─ [Service] nginx-web
│                       └─ NodePort 80 → 30000 → localhost:8080
│                   └─ [Deployment] prometheus (1 réplica)
│                       │   └─ ServiceAccount + ClusterRole + ClusterRoleBinding
│                           └─ ConfigMap con targets de scraping
│                               └─ kubernetes-pods (auto-descubrimiento)
│                                   └─ kubernetes-cadvisor (métricas de nodo)
│                   └─ [Service] prometheus
│                       └─ NodePort 9090 → 31000 → localhost:9090
│                   └─ [Deployment] grafana (1 réplica)
│                       └─ Datasource: http://prometheus:9090
│                   └─ [Service] grafana
│                       └─ NodePort 3000 → 32000 → localhost:3000
│                   └─ [NetworkPolicies] 5 reglas zero-trust
│                       └─ default-deny-all
│                       └─ allow-nginx-web
│                       └─ allow-monitoring-ingress (Prometheus + Grafana)
│                       └─ allow-prometheus-scrape
│                       └─ allow-dns
```

Puertos NAT (VirtualBox)

Servicio	Puerto Anfitrión (Host)	Puerto Invitado (VM)	Protocolo
SSH	2222	22	TCP
Nginx	8080	30000	TCP
Prometheus	9090	31000	TCP
Grafana	3000	32000	TCP

5. Fases de Implementación

5.1. Fase 1: Infraestructura y Aplicación Web

Responsable: Adrián Boza Suárez

Preparación del entorno

Se creó una máquina virtual en VirtualBox con Ubuntu Server 24.04 LTS, asignando 4 GB de RAM, 2 núcleos de CPU y 25 GB de almacenamiento dinámico. El adaptador de red se configuró en modo NAT con reglas de port forwarding para acceder a los servicios desde el host.

Instalación de los prerequisites:

```
# Instalar Docker
sudo apt update && sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER && newgrp docker

# Instalar Minikube

curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Instalar kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# Arrancar Minikube
minikube start --driver=docker --memory=2048 --cpus=2
```


Despliegue de Nginx

Fichero manifests/app-web/deployment.yaml :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-web
  labels:
    app: nginx-web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-web
  template:
    metadata:
      labels:
        app: nginx-web
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 10
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 5
          resources:
            requests:
              memory: "64Mi"
              cpu: "100m"
```

```
limits:
  memory: "128Mi"
  cpu: "250m"
```

Puntos clave del diseño:

- **2 réplicas:** Garantizan alta disponibilidad. Si un pod falla, el otro sigue sirviendo tráfico.
- **LivenessProbe:** Kubernetes verifica cada 10 segundos que Nginx responde HTTP 200 en `/`. Si falla 3 veces consecutivas, reinicia el pod automáticamente.
- **ReadinessProbe:** Kubernetes espera 5 segundos antes de enviar tráfico al pod, asegurando que Nginx está listo para recibir peticiones.
- **Resources limits:** 128 Mi RAM y 250 mCPU por pod, evitando que un pod sature los recursos de la VM.

Fichero `manifests/app-web/service.yaml` :

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-web
  labels:
    app: nginx-web
spec:
  type: NodePort
  selector:
    app: nginx-web
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30000
```

- **NodePort 30000:** Expone Nginx fuera del clúster. Se accede desde el host mediante NAT (8080 → 30000).
- **Selector `app: nginx-web` :** Enruta el tráfico a los pods con esa etiqueta, permitiendo que el Service balancee entre las 2 réplicas.

5.2. Fase 2: Monitorización

Responsable: Santiago Pérez Cano

Prometheus

Fichero `manifests/monitoring/prometheus.yaml` — contiene 6 recursos Kubernetes:

Recurso	Descripción
ServiceAccount prometheus	Identidad para los pods de Prometheus dentro del clúster
ClusterRole prometheus	Permisos de lectura sobre recursos del clúster (pods, services, endpoints, deployments) y acceso a <code>/metrics</code>
ClusterRoleBinding prometheus	Vincula la ServiceAccount con el ClusterRole
ConfigMap prometheus-config	Configuración de scraping con dos jobs: <code>kubernetes-pods</code> (auto-descubrimiento) y <code>kubernetes-cadvisor</code> (métricas del nodo)
Deployment prometheus	1 réplica con imagen <code>prom/prometheus:latest</code> , monta el ConfigMap como volumen
Service prometheus	NodePort 31000, selector <code>app=prometheus role=monitoring</code>

El scraping se realiza mediante dos mecanismos:

- 1. Auto-descubrimiento de pods (`kubernetes_pods`):** Prometheus consulta la API de Kubernetes para descubrir pods que tengan la anotación `prometheus.io/scrape: "true"` y los añade automáticamente como targets. Esto permite que cualquier nuevo pod con la anotación adecuada sea monitorizado sin configuración adicional.
- 2. cAdvisor (`kubernetes_cadvisor`):** Obtiene métricas de uso de CPU, memoria, disco y red del nodo a través del proxy de la API de Kubernetes, utilizando el endpoint `/metrics/cadvisor` de kubelet.

Grafana

Fichero `manifests/monitoring/grafana.yaml` — contiene 2 recursos:

Recurso	Descripción
Deployment grafana	1 réplica, imagen <code>grafana/grafana:10.4.3</code> , variables de entorno para login y bind a <code>0.0.0.0</code>
Service grafana	NodePort 32000, selector <code>app=grafana role=monitoring</code>

Configuración post-despliegue:

- Acceso a Grafana: `http://localhost:3000` con credenciales `admin / admin`
- Conexión del datasource Prometheus: URL `http://prometheus:9090` (nombre DNS interno del service de Kubernetes)

3. Importación del dashboard **Prometheus 2.0 Overview** (ID 3662), que proporciona gráficos preconfigurados de:

- Tasa de uso de CPU por pod y nodo
- Consumo de memoria RAM
- I/O de disco
- Tráfico de red entrante y saliente
- Estado de los targets de Prometheus

5.3. Fase 3: Seguridad de Red

Responsable: Sergio López Pérez

Se implementaron **5 NetworkPolicies** que siguen el modelo **zero-trust**: denegación total por defecto y apertura selectiva de puertos y protocolos únicamente para las comunicaciones necesarias.

Fichero `manifests/network/networkpolicy.yaml` :

Política 1: `default-deny-all`

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

- Afecta a **todos** los pods del namespace (`podSelector: {}`).
- Bloquea **todo** el tráfico entrante (Ingress) y saliente (Egress).
- Es la base del modelo zero-trust: nada está permitido a menos que otra política lo autorice explícitamente.

Política 2: `allow-nginx-web`

Permite tráfico entrante TCP al puerto 80 de los pods con etiqueta `app: nginx-web` desde cualquier origen (`from: []`). Esto permite el acceso a la aplicación web desde el exterior a través del NodePort.

Política 3: `allow-monitoring-ingress`

Dos políticas independientes que permiten tráfico entrante a:

- Prometheus (TCP 9090) — para acceder a la interfaz web de Prometheus
- Grafana (TCP 3000) — para acceder al dashboard de Grafana

Política 4: `allow-prometheus-scrape`

Permite que los pods con etiqueta `app: prometheus` inicien conexiones salientes TCP al puerto 80 de pods con `app: nginx-web` . Esto es necesario para que Prometheus pueda recoger métricas de Nginx.

Política 5: `allow-dns`

Permite tráfico DNS (UDP 53 y TCP 53) saliente desde cualquier pod. Sin esta política, los pods no pueden resolver nombres de servicios (`prometheus:9090`) ni dominios externos.

6. Resultados y Verificación

6.1. Despliegue de la aplicación web

```
adrian@servidoradrian:~$ kubectl get all -l app=nginx-web
NAME                                READY    STATUS    RESTARTS    AGE
pod/nginx-web-67c5cdbd5c-jn2df      1/1     Running   1 (14m ago)  67m
pod/nginx-web-67c5cdbd5c-xgqlm      1/1     Running   1 (14m ago)  67m

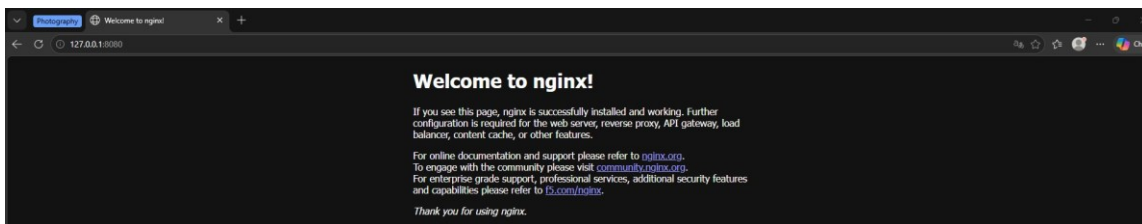
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
service/nginx-web                   NodePort    10.105.237.126 <none>         80:30000/TCP    67m

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/nginx-web          2/2      2              2            67m

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/nginx-web-67c5cdbd5c 2          2          2        67m
adrian@servidoradrian:~$
```

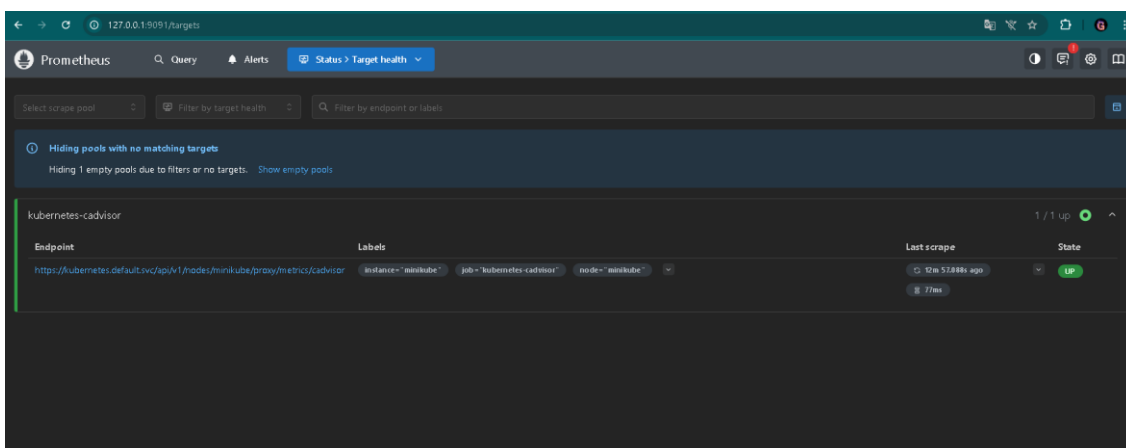
Los pods de Nginx aparecen en estado `Running` con `1/1` containers listos, confirmando que el Deployment se ha creado correctamente con las 2 réplicas configuradas.

6.2. Acceso desde el navegador



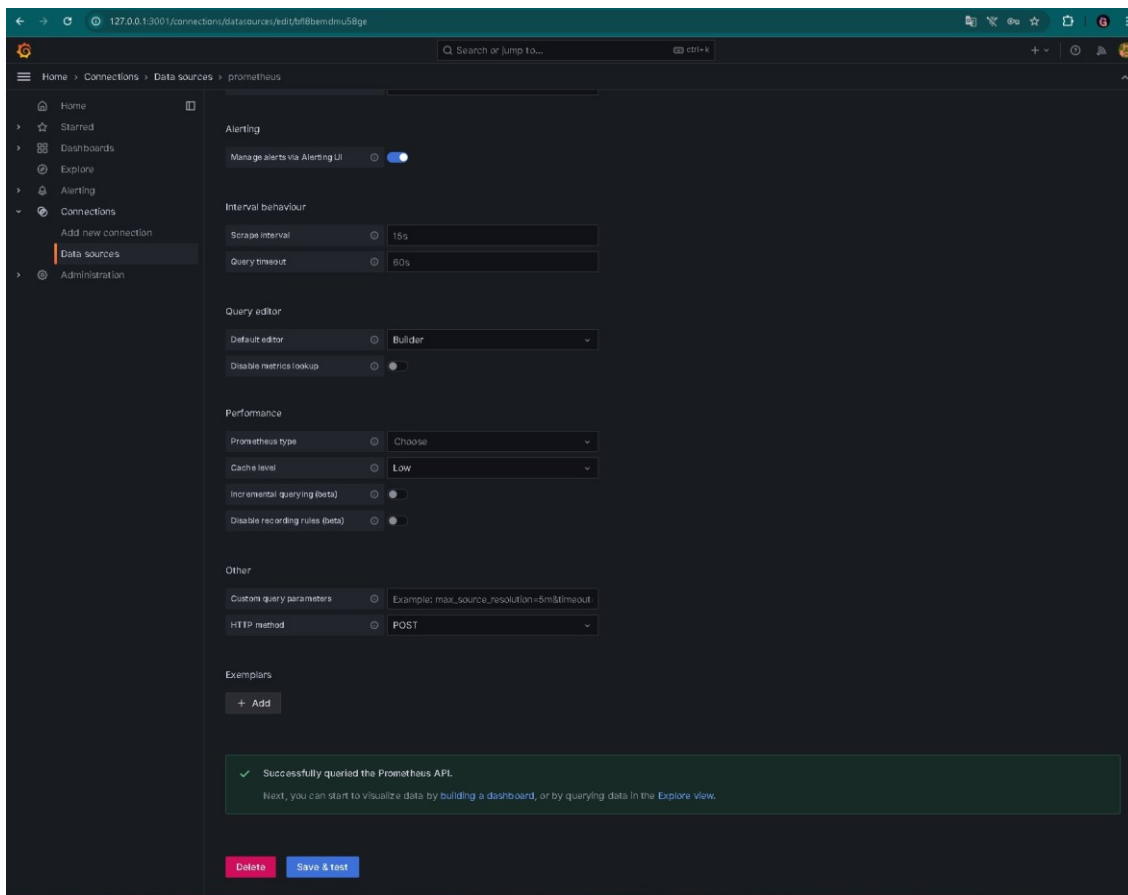
La aplicación web Nginx responde correctamente accediendo a `http://localhost:8080` desde el navegador del host, mostrando la página de bienvenida por defecto de Nginx.

6.3. Prometheus targets UP



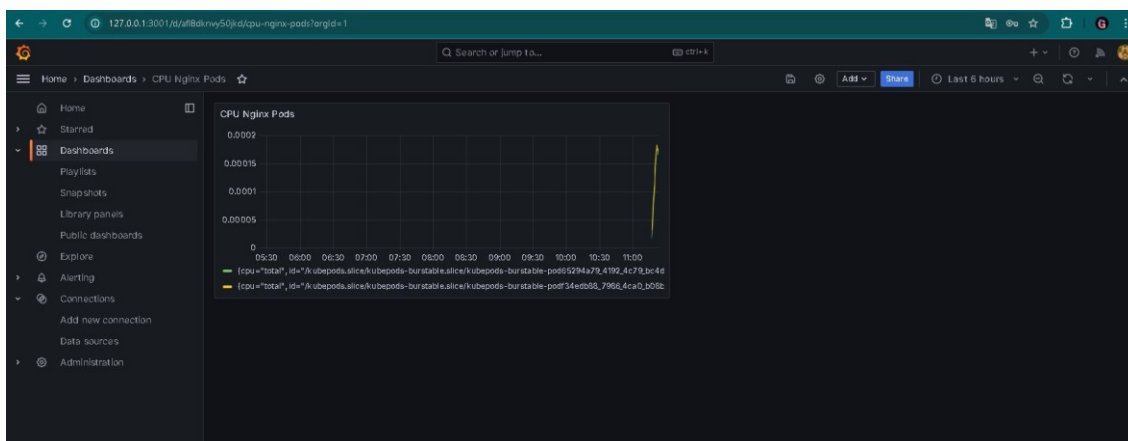
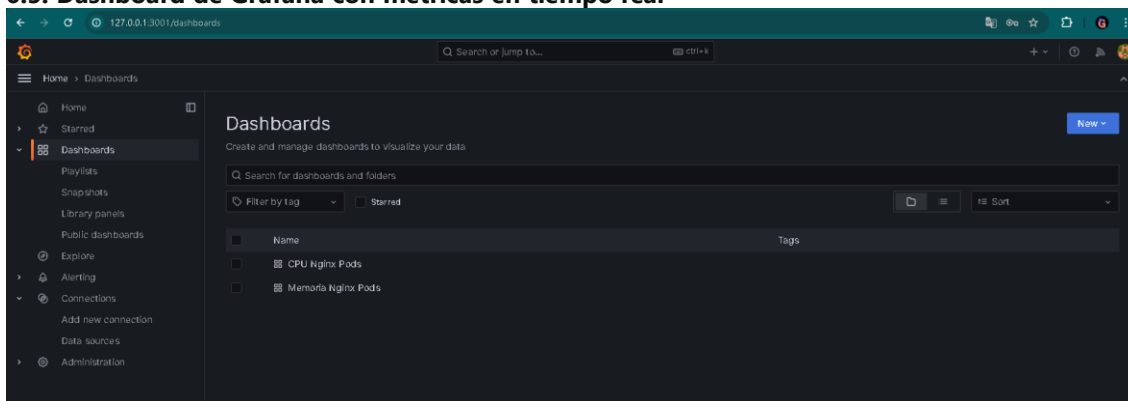
Prometheus muestra todos los targets configurados en estado `UP` (verde), confirmando que el scraping funciona correctamente tanto para pods como para cAdvisor.

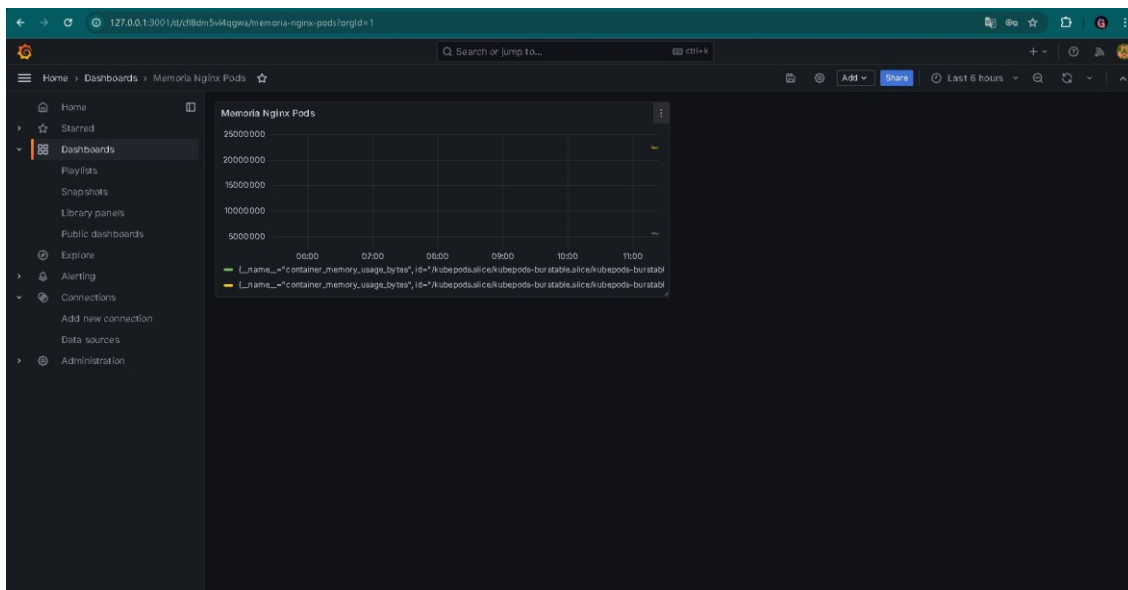
6.4. Grafana con datasource conectado



El datasource de Prometheus en Grafana responde "Data source is working", confirmando la conectividad entre Grafana y Prometheus a través del DNS interno del clúster.

6.5. Dashboard de Grafana con métricas en tiempo real





El dashboard **Prometheus 2.0 Overview** (ID 3662) muestra métricas en tiempo real del clúster, incluyendo uso de CPU, memoria, I/O de disco y tráfico de red.

6.6. NetworkPolicies aplicadas

```
sergio@ubuntu-sergio:~$ kubectl get networkpolicy
NAME                                POD-SELECTOR    AGE
allow-dns                          <none>          20h
allow-grafana-ingress               app=grafana     20h
allow-monitoring-ingress            app=prometheus  20h
allow-nginx-web                     app=nginx-web   20h
allow-prometheus-scrape              app=prometheus  20h
default-deny-all                   <none>          20h
sergio@ubuntu-sergio:~$
```

Se confirma la aplicación de las 5 políticas de red en el namespace `default`.


```
sergio@ubuntu-sergio:~$ kubectl describe networkpolicy default-deny-all
Name:          default-deny-all
Namespace:     default
Created on:    2026-05-19 18:31:12 +0000 UTC
Labels:        app=network-policy
               role=security
Annotations:   <none>
Spec:
  PodSelector:    <none> (Allowing the specific traffic to all pods in this namespace)
  Allowing ingress traffic:
    <none> (Selected pods are isolated for ingress connectivity)
  Allowing egress traffic:
    <none> (Selected pods are isolated for egress connectivity)
  Policy Types:   Ingress, Egress
sergio@ubuntu-sergio:~$
```

Detalle de las reglas de cada política, verificando que los selectores y puertos son correctos.

6.7. Test de conectividad



Se verificó mediante pruebas prácticas que:

- Nginx es accesible desde el exterior a través del NodePort
- Prometheus puede scrapear métricas de Nginx
- Grafana puede consultar a Prometheus para obtener datos
- El tráfico no autorizado está bloqueado por las NetworkPolicies

6.8. Prueba de alta disponibilidad

```
# Comando ejecutado:
kubectl delete pod -l app=nginx-web --force

# Resultado:

# El pod eliminado fue recreado automáticamente por el ReplicaSet en < 30 segundos
# Sin pérdida de servicio (el otro pod seguía funcionando)
```

Esta prueba demuestra el **auto-healing** de Kubernetes: el ReplicaSet detecta que el número de réplicas real (1) es inferior al deseado (2) y crea un nuevo pod automáticamente

7. Problemas Encontrados y Soluciones

Problema	Causa	Solución	Afectó a
Minikube no arrancaba	RAM insuficiente (1 GB)	Aumentar a 2 GB con <code>--memory=2048</code>	Adrián
Prometheus no scrapeaba Nginx	NetworkPolicy bloqueaba el tráfico de scraping	Añadir política <code>allow-prometheus-scrape</code> con Egress TCP 80 hacia <code>app: nginx-web</code>	Santiago
<code>kubect1 apply -f manifests/</code> daba error	Faltaba flag <code>-R</code> para incluir subdirectorios	Usar <code>kubect1 apply -f manifests/ -R</code>	Sergio
ERR_CONNECTION_REFUSED en navegador	NAT de VirtualBox no configurado	Configurar reenvío de puertos en VirtualBox (8080→30000, 9090→31000, 3000→32000)	Todo el equipo
Dashboard de Grafana sin datos	Dashboard 3662 usa métricas internas de Prometheus, no de Nginx	Usar modo Explore con <code>query up</code> para verificar conectividad y datos reales	Santiago
Pods en estado <code>Pending</code>	Recursos insuficientes en la VM	Revisar con <code>kubect1 describe pod</code> y ajustar <code>Resources limits</code> en los YAMLS	Adrián
Disco del host Windows lleno	Espacio insuficiente en C:	Migrar la VM al disco E: con más capacidad	Sergio
Clave SSH cambiada tras reinstalación	Host key mismatch por regeneración de claves	<code>ssh-keygen -R [127.0.0.1]:2222</code> para limpiar la clave antigua	Todo el equipo

8. Relación con los Módulos de ASIR

Servicios de Red e Internet (SRI)

RA	Cómo se aplica en el proyecto
RA1 — Servicios de Red	Configuración de Services en Kubernetes (NodePort) para exponer Nginx, Prometheus y Grafana. Configuración del reenvío de puertos NAT en VirtualBox.
RA2 — Servicios Web	Implantación de un servidor web Nginx contenerizado con health checks (livenessProbe + readinessProbe).
RA4 — Alta Disponibilidad	Uso de 2 réplicas de Nginx con auto-recuperación mediante ReplicaSet y probes de salud.

Seguridad y Alta Disponibilidad (SAD)

RA	Cómo se aplica en el proyecto
RA1 — Seguridad en Redes	Configuración de 5 NetworkPolicies para aislamiento de tráfico entre pods siguiendo el modelo zero-trust.
RA3 — Continuidad del Negocio	Resiliencia mediante auto-recuperación de pods (self-healing) y alta disponibilidad con 2 réplicas que garantizan servicio ininterrumpido.
RA5 — Recuperación	Gestión del estado del clúster: si un pod falla, Kubernetes lo recrea automáticamente sin intervención manual.

Administración de Sistemas Operativos (ASO)

RA	Cómo se aplica en el proyecto
RA2 — Gestión de Recursos	Monitorización de CPU y RAM mediante Prometheus + Grafana. Limitación de recursos por pod (128 Mi RAM / 250 mCPU).
RA4 — Virtualización	Despliegue de infraestructura sobre Docker y orquestación con Minikube dentro de una VM VirtualBox.
RA6 — Automatización	Gestión de infraestructura como código mediante manifiestos YAML declarativos versionados en Git.

9. Conclusiones

Logros alcanzados

1. **Entorno funcional y accesible:** Se ha implementado un clúster Kubernetes totalmente operativo en un portátil con recursos limitados, demostrando que la tecnología cloud-native es accesible incluso en entornos educativos sin presupuesto.
2. **Alta disponibilidad demostrada:** La aplicación web Nginx con 2 réplicas y health checks garantiza que el servicio se mantiene incluso ante fallos de un pod, con recuperación automática en menos de 30 segundos.
3. **Monitorización completa:** Prometheus recopila métricas del clúster mediante auto-descubrimiento y Grafana las visualiza en dashboards profesionales, proporcionando observabilidad en tiempo real.
4. **Seguridad zero-trust:** Las 5 NetworkPolicies implementan un modelo de seguridad donde todo el tráfico está denegado por defecto y solo se permiten comunicaciones explícitamente autorizadas.
5. **Infraestructura como código:** Todos los manifiestos están versionados en Git, garantizando reproducibilidad total del entorno. Cualquier persona puede clonar el repositorio y tener el mismo clúster funcionando.

Lecciones aprendidas

- Las NetworkPolicies deben planificarse cuidadosamente: un orden incorrecto o reglas demasiado restrictivas pueden bloquear servicios necesarios como DNS o el scraping de Prometheus.
- La limitación de recursos es crítica en entornos con restricciones de hardware. Sin `resources.limits` pod puede saturar la VM y provocar caídas del clúster.
- El auto-descubrimiento de Prometheus mediante anotaciones simplifica enormemente la configuración de scraping y es una práctica recomendada en producción.
- La documentación detallada con capturas de pantalla y comandos exactos es fundamental tanto para la evaluación como para la reproducción del proyecto por parte de terceros.
- El trabajo en equipo con Git requiere coordinación: establecer convenciones de nombres, commits descriptivos y revisión cruzada de los manifiestos.

10. Mejoras Futuras

Mejora	Prioridad	Impacto	Descripción
Horizontal Pod Autoscaler (HPA)	Alta	Alto	Escalar automáticamente Nginx de 2 a 5 réplicas si la CPU supera el 70 %.
Persistent Volume Claims (PVC)	Alta	Alto	Retener datos de Prometheus y dashboards de Grafana tras reinicios de pods.
Ingress Controller + HTTPS	Alta	Alto	Reemplazar NodePort por Nginx Ingress con certificados Let's Encrypt para acceso HTTPS.
Annotations para Prometheus	Media	Alto	Añadir <code>prometheus.io/scrape: "true"</code> en el deployment de Nginx para discovery automático.
Namespaces dedicados	Media	Medio	Separar recursos en namespaces <code>app-web</code> , <code>monitoring</code> y <code>network</code> para aislamiento lógico.
Pipeline CI/CD (GitHub Actions)	Media	Alto	Validar YAMLS con kubeconform y desplegar automáticamente tras push a main.
Logging centralizado (Loki)	Baja	Medio	Agregar logs de todos los pods en un dashboard único de búsqueda con Grafana Loki.
Cluster multi-nodo (k3s)	Baja	Crítico	Migrar a k3s con varios nodos para tolerancia real a fallos.

11. Anexo: Evidencias

Índice de capturas

#	Archivo	Descripción	Fase
1	03-nginx-deployment.png	Pods de Nginx en estado Running con 2 réplicas	App Web
2	04-nginx-browser.png	Página de bienvenida de Nginx desde el navegador del host	App Web
3	05-prometheus-targets.png	Targets de Prometheus en estado UP	Monitoring
4	06-grafana-datasource.png	Conexión del datasource Prometheus en Grafana	Monitoring
5	06.2-grafana-datasource.png	Verificación del datasource: "Data source is working"	Monitoring
6	07-grafana-dashboard.png	Dashboard Prometheus 2.0 Overview con métricas de CPU	Monitoring
7	07.2-grafana-dashboard.png	Dashboard con métricas de memoria y disco	Monitoring
8	07.3-grafana-dashboard.png	Dashboard con tráfico de red	Monitoring
9	09-networkpolicy-list.png	Lista de las 5 NetworkPolicies aplicadas	Seguridad
10	10-networkpolicy-describe.png	Detalle de una NetworkPolicy con sus reglas	Seguridad
11	11-conectividad-test.png	Pruebas de conectividad entre servicios	Seguridad

Comandos de verificación utilizados

```
# Ver estado del clúster
kubectl get pods -o wide
kubectl get svc
kubectl get networkpolicies

# Ver detalles de un recurso
kubectl describe pod -l app=nginx-web
kubectl describe networkpolicy allow-nginx-web

# Probar conectividad interna
kubectl exec -it deployment/prometheus -- wget -qO- http://nginx-web:80 | head -5

# Prueba de alta disponibilidad
kubectl delete pod -l app=nginx-web --force
kubectl get pods -w

# Acceder a la app desde la VM
curl http://$(minikube ip):30000
```

12. Anexo: Glosario de Términos

Término	Definición
Pod	Unidad mínima de despliegue en Kubernetes. Contiene uno o más contenedores que comparten red y almacenamiento.
Deployment	Recurso que gestiona un conjunto de pods idénticos, garantizando el número de réplicas deseado y permitiendo actualizaciones rolling.
Service	Abstracción que expone un conjunto de pods como un servicio de red estable con IP y DNS propios.
NodePort	Tipo de Service que expone el puerto del pod en un puerto del nodo (rango 30000–32767), accesible desde fuera del clúster.
NetworkPolicy	Recurso que define reglas de firewall a nivel de pod, controlando el tráfico Ingress (entrante) y Egress (saliente).
Namespace	Mecanismo de aislamiento lógico dentro del clúster para organizar recursos y aplicar cuotas.
ConfigMap	Recurso para almacenar configuración no sensible en forma de pares clave-valor o ficheros.
RBAC	Role-Based Access Control. Sistema de control de acceso basado en roles para la API de Kubernetes.
ServiceAccount	Identidad para pods que permite autenticarse contra la API de Kubernetes.
ClusterRole	Conjunto de permisos a nivel de clúster (no limitado a un namespace).
Minikube	Herramienta que ejecuta un clúster Kubernetes de un único nodo en local para desarrollo y pruebas.
kubectl	CLI oficial para interactuar con la API de Kubernetes desde la línea de comandos.
LivenessProbe	Sonda que determina si un contenedor está vivo. Si falla, Kubernetes reinicia el contenedor.
ReadinessProbe	Sonda que determina si un contenedor está listo para recibir tráfico. Si falla, el pod se retira del Service.
Prometheus	Sistema de monitoreo y alerta de código abierto que recopila métricas mediante scraping HTTP.
Grafana	Plataforma de visualización de métricas con dashboards interactivos y configurables.
cAdvisor	Agente integrado en kubelet que expone métricas de uso de recursos del nodo y los contenedores.
Zero-trust	Modelo de seguridad donde ningún tráfico es confiable por defecto; todo debe autenticarse y autorizarse explícitamente.

Auto-healing	Capacidad de Kubernetes para detectar y recuperar automáticamente pods fallidos sin intervención manual.
IaC (Infrastructure as Code)	Práctica de gestionar la infraestructura mediante archivos de configuración declarativos versionados en Git.

Autores: Adrián Boza Suárez, Santiago Pérez Cano, Sergio López Pérez

Repositorio: <https://github.com/adrianboza2/proyecto-intermodular-asir>